

基于 RV1126B 的宿舍公共工具智能借还管理终端

摘要

本作品面向高校宿舍楼层、班级活动室等公共场景中常用工具（螺丝刀、剪刀、直尺、胶带、订书机等）普遍存在的“借还登记不规范、归还不及时、缺失难发现”等痛点，设计并实现了一套基于瑞芯微 RV1126B 高性能 AI 处理器的宿舍公共工具智能借还管理终端。系统硬件以 RV1126B 开发板为核心，集成 MIPI OV13855 人脸识别摄像头、USB 物品识别摄像头、7 英寸 MIPI 触摸屏，充分利用 RV1126B 片上 NPU 的边缘算力，实现“身份验证—物感监控—Web 端闭环管理”的全流程自动化。

在算法层面，系统采用双路 RKNN 模型并行推理架构：一路以 RetinaFace 完成人脸检测与五点关键点定位，经仿射变换完成人脸对齐后，由 MobileFaceNet 提取特征向量，通过欧式距离比对实现非接触式身份验证，并针对开源人脸数据集的人种偏差问题，采用自建的亚洲人脸数据集进行微调；另一路基于自采集数据集微调训练的 YOLOv8s 模型，借助多线程 RKNN 推理池在 NPU 上实时检测绷带、胶棒、直尺、剪刀、螺丝刀、订书机、胶带等 7 类工具的槽位在位状态。系统后端依托 Flask 与 SQLite 构建轻量级业务层，通过 MJPEG 协议与非阻塞式帧缓冲区锁机制，将双路视频流与检测叠加结果实时推送至 Chromium Kiosk 全屏前端；交互设计中引入“AI 识别 + 人工二次确认”的容错模式，保障了业务的高可靠性。同时提供新用户录入、借用/归还、报损与丢失上报、库存增减、借还日志查询等完整业务闭环，学生与管理员分别拥有不同的操作权限与借用规则约束。

经数据集验证，物品识别模型整体检测精度 mAP50 达 97.8%，各类别 mAP50-95 均值约 67.5%；人脸识别模块采用双模型级联方案，在保证较高实时性的同时兼顾了识别准确率。本作品实现了边缘 AI 视觉识别技术与宿舍公共物品管理业务场景的深度融合，为解决高校及公共场所内共享工具流转不透明、管理效率低的问题，提供了一套低成本、易复制、可扩展的边缘智能终端解决方案。

第一部分 作品概述

1.1 功能与特性

1、实时人脸身份验证：

基于 RetinaFace + MobileFaceNet 级联流水线，对进入识别区域的人员进行实时检测、五点对齐与特征比对，无需刷卡或按键即可完成身份识别，2.5 秒容错窗口有效避免因抖动、遮挡导致的身份误判；

2、多类别工具检测：

基于自建数据集微调的 YOLOv8s 模型，实时识别借用台上的绷带、胶棒、直尺、剪刀、螺丝刀、订书机、胶带 7 类工具，并统计当前画面中各类物品数量；

3、双路视频网页推流：

终端以 Flask + SQLite 构建轻量级业务后端，通过 MJPEG 协议将人脸识别画面与物品识别画面双路视频流与检测叠加结果推送至 Chromium 全屏 Kiosk 前端，使用人员可通过触控显示屏，通过推流网页进行实时交互；

4、借用/归还/报损全流程记录：

学生界面：直观展示当前已借用物资、应还时间，并提供“正常归还”、“报损”、“丢失”一键申报入口；

管理员界面：集成实时库存调整、异常报损清单审批、全量借还历史审计等功能；

1.2 应用领域

本系统面向存在高频信息录入与需求、传统人工登记方式效率低下、纠纷追溯困难且缺乏专人值守的场景，具有较强的通用性和可复制性，主要应用领域包括：

1、高校宿舍楼层公共工具柜：面向学生日常维修、手工制作等场景下的螺丝刀、剪刀、胶带等公共物资的自助借还与溯源，减少物品遗失与纠纷。如图 1.2 展示宿舍一站式便生服务站的应用场景：



图 1.2 应用场景图

2、无人值守微型库房：共享打印耗材、办公文具、维修工具等低值易耗品的自助领用与库存盘点。

3、智能门禁管理系统：结合实时人脸识别功能，扩展用于宿舍安全准入与考勤数据联动。

1.3 主要技术特点

1、级联式人脸识别与几何仿射对齐：

采用 RetinaFace 检测定位结合仿射变换实现五点人脸关键点标准化对齐，再输入 MobileFaceNet 提取特征。

同时，利用自建特定族群数据集进行微调，有效克服了边缘端人脸偏转、姿态多变带来的相似度漂移以及算法的人种泛化偏差。

2、多线程推理池与负载均衡：

基于 rknnPoolExecutor 部署多核心（TPEs=4）异步并行推理池。通过流水线式设计实现图像采集、推理与后处理并发执行，隐藏数据交互延迟，提升系统吞吐；

3、Flask 非阻塞式双路并发推流：

人脸识别与物品识别分别运行在独立线程中，通过 MJPEG 协议将处理后的画面实时推送至网页端，互不阻塞。媒体层辅以非阻塞式帧缓冲区锁机制，确保 Web 网页端 MJPEG 视频流帧率不受 NPU 推理耗时波动的干扰。

4、Web 端反馈二次确认机制：

在 Web 交互设计中引入“AI 识别 + 人工二次确认”的容错模式，允许用

户在提交前修正 AI 误检结果，兼顾自动化效率与业务可靠性；

1.4 主要性能指标

表 1.4.1 系统核心性能指标与实测结果汇总表

性能指标	实测结果	备注说明
人脸识别准确率	> 95%	RetinaFace + MobileFaceNet 余弦相似度阈值 0.95
物品识别整体 mAP50	97.8%	基于 yolov8s.pt 与自建数据集
物品识别单帧延迟	< 15 ms	YOLOv8 基于 4 线程 RKNN 推理池加速
系统端到端推流延迟	< 80 ms	Web M-JPEG 推流，帧率限制在 30 FPS
支持识别物资类别	7 类	剪刀、胶带、绷带、螺丝刀、胶棒、直尺、订书机

表 1.4.2 基于 YOLOv8 的多类别物资检测性能评估表

类别	mAP@0.5	识别召回率
Bandage (绷带)	0.982	0.932
Glue stick (胶棒)	0.929	0.954
Ruler (直尺)	0.995	0.987
Scissors (剪刀)	0.969	0.884
Screwdriver (螺丝刀)	0.990	0.973
Stapler (订书机)	0.986	0.968
Tape (胶带)	0.995	1.00
全类别	0.978	0.957
整体 Precision(B) / Recall(B)	0.973 / 0.957	—

1.5 主要创新点

1、级联模型设计与算法偏差校正：

采用“检测+对齐+提取”双模型级联架构，利用关键点仿射变换克服多角度姿态偏转导致的识别漂移；并基于自建亚洲人脸数据集微调特征网络，提升特定族群分类边界清晰度，有效缓解开源模型的人种泛化偏差；

2、基于 RKNN 多线程双路模型并行调度：

深度挖掘 RV1126B 端侧算力，基于 rknnPoolExecutor 部署多核（TPEs=4）异步并行推理池，采用两个常驻线程独立进行图像采集与 NPU 推理，避免了 I/O 阻塞对 Web 服务响应造成的影响，兼顾了实时性与识别准确率，保障了双模型并发的高吞吐与低延迟

3、软硬件一体化闭环设计：

首创将端侧多目标物品清点与管理规则强耦合，设计了人脸离域自锁与“AI

检测+人工微调”容错机制，实现了集刷脸登录、库存自动扣减、异常报损申报于一体的软硬件端到端闭环。

1.6 设计流程

1、需求定义与硬件选型：明确身份鉴权与物资清点需求，选定 RV1126B 核心板，完成双路摄像头、触摸屏及结构件的适配与连接。

2、模型微调与端侧编译：基于自建数据集微调训练人脸识别与 YOLOv8s 模型，编写相关转换代码，执行.pt 到.onnx 到.rknn 的编译转换与量化优化。

3、并发程序与 Web 开发：搭建多线程推理框架并基于 Flask + SQLite 构建网页与数据库后台，实现图像处理与前端 MJPEG 的低延迟推流交互。

4、全链联调与性能寻优：开展软硬件联合调试与测试，针对检测精度、推理耗时和界面响应速度进行寻优调整，完成最终部署方案。

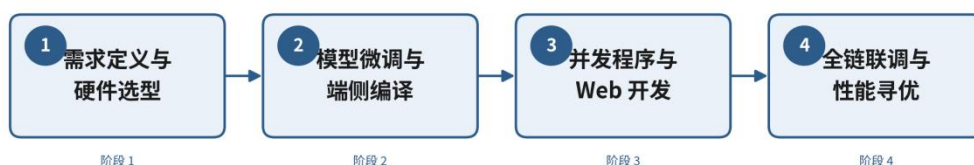


图 1.6 设计流程图

第二部分 系统组成及功能说明

2.1 整体介绍

系统由感知层、算力层、业务层与交互层四部分组成：感知层由人脸识别摄像头与物品识别 USB 摄像头构成，分别采集身份核验画面与工具放置画面；算力层依托 RV1126B 内置 NPU，通过 RKNNLite Runtime 承载 RetinaFace/MobileFaceNet 与 YOLOv8 两条推理流水线，并以多线程推理池提升吞吐；业务层基于 Flask 实现身份判定、借还状态机与数据库读写；交互层以 Chromium Kiosk 全屏界面呈现识别画面、借还清单确认弹窗与管理后台。四层之间以“视频帧→推理结果→业务状态→界面渲染”的单向数据流贯通，各模块通过线程安全的共享状态对象与互斥锁解耦，保证高并发场景下数据一致性。系统总体架构如图 2.1 所示。



图 2.1 系统框图

2.2 硬件系统介绍

2.2.1 硬件整体介绍

系统硬件以 RV1126B 开发板为主控核心，搭载两路摄像头：一路 OV13855 摄像头通过 MIPI CSI 接口连接开发板用于人脸采集；另一路通 USB 摄像头用于物品识别。开发板通过 MIPI DSI 接口驱动 7 英寸触摸屏用于本地人机交互；同时开发板通过烧写、调试串口与网线与笔记本电脑相连。

2.2.2 机械设计介绍

机械结构设计以“工具存放仓 + 物品检测平台 + 显示屏交互窗口”三段式为总体思路：USB 摄像头置于检测平台上方，俯视覆盖工具存放平台；中层为工具分格存放槽位，便于按类别摆放各类工具；主控板与显示屏交互窗口嵌在外壳前部，人脸识别摄像头通过下方显示屏下方小孔捕捉人脸。整体结构设计使用三维建模软件完成尺寸标注与线框校验，如图 2.2.1 所示；图 2.2.2 为对应的三维渲染效果图。

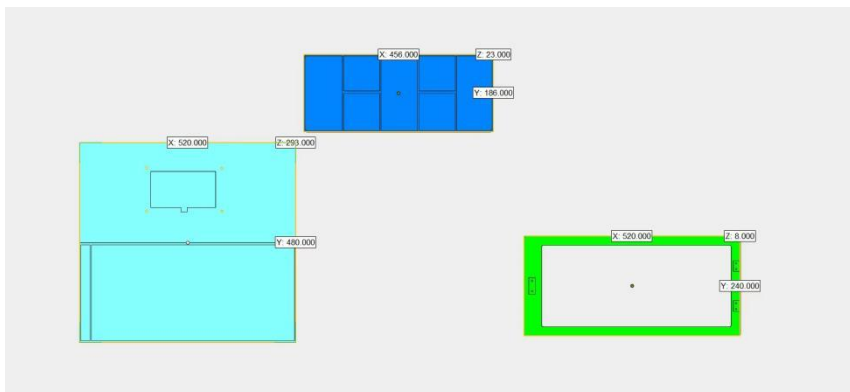


图 2.2.1 机械结构三维线框与尺寸标注图

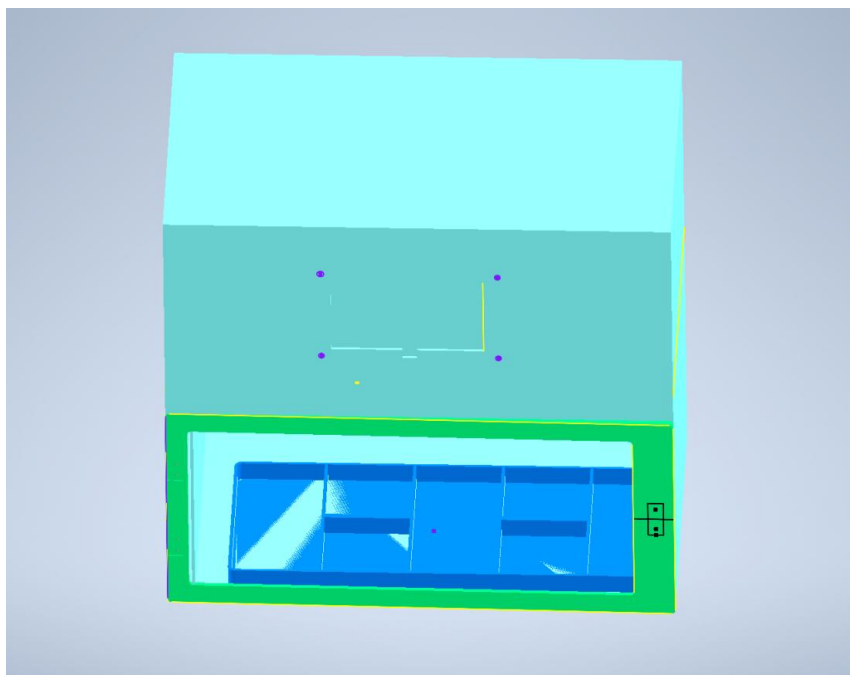


图 2.2.2 机械结构三维渲染效果图

2.3 软件系统介绍

2.3.1 软件整体介绍

运行环境为开发板载 Linux 系统，主程序 main_app.py 以 Flask 应用为核心，启动时加载 RetinaFace、MobileFaceNet 两个 RKNN 模型并完成 NPU 运行时初始化，随后拉起 inference_thread（人脸识别）与 yolo_inference_thread（物品识别）两个后台线程并行工作。软件核心框架如下：

软件核心框架


```

/userdata/SmartStorage_RV1126B/
├── main_app.py                # 主程序
├── database.py                # 数据库表结构初始化脚本
├── smart_storage.db           # 运行时生成的 SQLite 数据库
├── func/
│   └── func_yolov8_optimize.py # YOLOv8 后处理算子优化与处理函数
├── rknnModel/
│   └── best.rknn              # NPU YOLOv8 物品识别模型
├── rknnpool/
│   └── rknnpool_ld.py         # RKNN 线程池创建与并发调度管理脚本
├── models/
│   ├── RetinaFace.rknn        # NPU 级联人脸检测模型
│   ├── MobileFaceNet.rknn     # NPU 人脸特征提取模型
│   ├── RetinaFace_mobile320.onnx
│   ├── mobilefacenet.onnx
│   └── best.rknn              # YOLOv8 物品识别模型备份
├── templates/
│   ├── index.html             # 智能仓库主页
│   ├── register.html          # 身份录入页面
│   └── dashboard.html         # 业务控制台
└── static/
    ├── index.css              # 前端全局样式表
    └── index.js                # 虚拟键盘事件绑定、数据轮询与网络请求逻辑
  
```

2.3.2 软件各模块介绍

1、人脸检测与识别模块

级联执行 RetinaFace 检测框/关键点解码、仿射对齐与 MobileFaceNet 特征比对，核心逻辑如下：

```

def align_face(img, ldm):
    dst = np.array([[38.29, 51.69], [73.53, 51.50], [56.02, 71.73],
                    [41.54, 92.36], [70.72, 92.20]], dtype=np.float32)
    src = np.array([[ldm[0], ldm[1]], [ldm[2], ldm[3]], [ldm[4], ldm[5]],
                    [ldm[6], ldm[7]], [ldm[8], ldm[9]]], dtype=np.float32)
    tform, _ = cv2.estimateAffinePartial2D(src, dst)
    return cv2.warpAffine(img, tform, (112, 112))

# main_app.py • inference_thread()
if len(keep) > 0:
    best_idx = idx[keep[0]]
    face = align_face(frame, landms[best_idx])
    feat = rec_sess.inference(inputs=[preprocess_rec(face))][0][0]
    feat /= np.linalg.norm(feat)
    gs.latest_feature = feat
  
```




```
for user in gs.face_db:
    dist = np.linalg.norm(feats - user["feat"])
    if dist < min_dist:
        min_dist = dist
    if dist < 0.95:
        temp_user = {"name": user["name"], "role": user["role"],
                     "id": user["id"]}
```

2、物品检测模块

基于自训练 YOLOv8 模型，采用 DFL 解码还原检测框坐标，并按类别分组执行 NMS，抑制冗余检测框

```
def dfl(position): # Distribution Focal Loss 解码
    n, c, h, w = position.shape
    p_num, mc = 4, c // 4
    y = position.reshape(n, p_num, mc, h, w)
    e_y = np.exp(y - np.max(y, axis=2, keepdims=True))
    y = e_y / np.sum(e_y, axis=2, keepdims=True)
    acc_matrix = np.arange(mc).reshape(1, 1, mc, 1, 1)
    return (y * acc_matrix).sum(2)

def myFunc(rknn_lite, IMG): # 推理池回调：单帧检测+绘制+计数
    IMG2 = cv2.cvtColor(IMG, cv2.COLOR_BGR2RGB)
    IMG2, ratio, padding = letterbox(IMG2)
    outputs = rknn_lite.inference(inputs=[np.expand_dims(IMG2, 0)],
                                   data_format=['nhwc'])
    boxes, classes, scores = yolov8_post_process(outputs)
    counts = {cls: 0 for cls in CLASSES}
    if boxes is not None:
        draw(IMG, boxes, scores, classes, ratio, padding)
        for c in classes:
            counts[CLASSES[int(c)]] += 1
    return IMG, counts
```

3、多线程推理池调度模块

借鉴生产者-消费者模型，主循环持续 pool.put(frame) 提交待推理帧，异步从 pool.get() 取回已完成推理的结果帧与计数字典，4 个工作线程（TPEs=4）共享同一 RKNN 模型实例并行处理，在保证画面连续性的同时提升检测吞吐，避免单线程推理阻塞视频采集导致的画面卡顿

4、Web 业务与数据库模块

以 SQLite 存储用户、工具库存与借还记录三张核心数据表

```

CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT, role TEXT,
                     dorm_info TEXT, password TEXT, feature BLOB);
CREATE TABLE items (id INTEGER PRIMARY KEY, name TEXT, count INTEGER);
CREATE TABLE records (id INTEGER PRIMARY KEY, user_id INTEGER, item_id
                       INTEGER,
                       borrow_time TIMESTAMP, due_time TIMESTAMP, status TEXT,
                       admin_note TEXT DEFAULT 'unprocessed');
  
```

5、前端交互模块

首页每秒轮询 /get_current_user 接口，根据当前识别到的身份动态显示“陌生人/欢迎XXX”状态及进入按钮；
 学生工作台通过 /video_feed_usb 展示物品识别画面，点击“确定借用”后调用 /api/get_counts 拉取当前检测计数并以可编辑表单弹窗展示，支持人工修正后再提交 /do_borrow_batch；
 管理员后台展示实时库存、待处理报损清单与全量借还历史；注册页面集成虚拟键盘（Simple-Keyboard），适配无实体键鼠的触屏一体化终端形态。

第三部分 完成情况及性能参数

3.1 整体介绍

系统实物图如图 3.1.1 及图 3.1.2 所示。



图 3.1.1 实物正面图



图 3.1.2 实物斜 45° 图

3.2 工程成果

3.2.1 机械成果

通过 3D 建模设计外壳，结合制造技术的成本以及尺寸考虑，采用木工制作方式，制作了外壳以及放置物品的托盘。每部分外观实物图如图 3.2.1.1、图 3.2.1.2 所示。



图 3.2.1.1 外壳外观图



图 3.2.1.2 物置托盘图

3.2.2 软件成果

本系统软件界面依托 Chromium Kiosk 全屏网页呈现，构建了高交互性的闭环管理后台：

（1）学生工作台：

支持无感刷脸快速登录(图 3.2.2.1)与借用台物品实时框选识别(图 3.2.2.2)。系统设计了“AI 检测 + 人工修正”二次确认弹窗(图 3.2.2.3)，以修正边缘端误检；同时具备安全保护机制，一旦人脸离开识别区域即自动弹窗拦截，防止越权操作(图 3.2.2.4)。



图 3.2.2.1 开发板人脸识别界面



图 3.2.2.2 物品借用识别界面

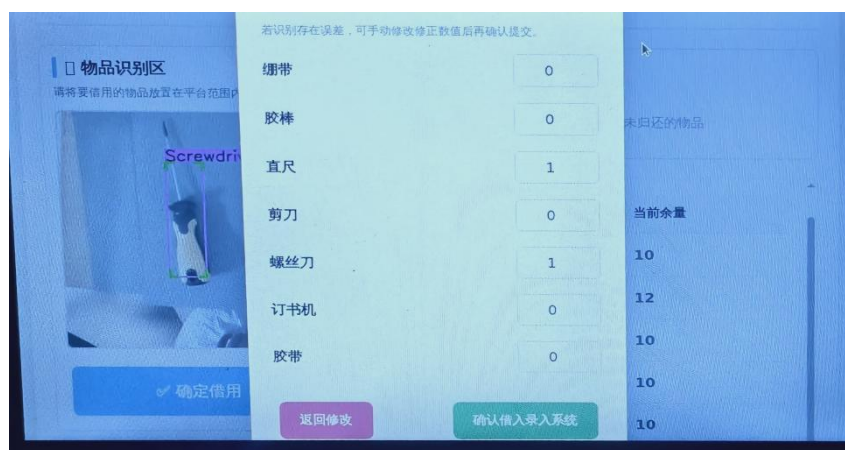


图 3.2.2.3 人工二次确认各种类物品借用界面



图 3.2.2.4 人脸离开识别区域则取消操作权限提示界面

(2) 管理员后台:

管理员通过人脸验证登录后（图 3.2.2.5），可对库存余量进行一键快捷调整、对学生上报的异常丢失/损坏清单进行在线审批（图 3.2.2.6），并能审计全量借还历史记录（图 3.2.2.7），实现了精细化、可追溯的数据后台管理。



图 3.2.2.5 管理员识别登录界面

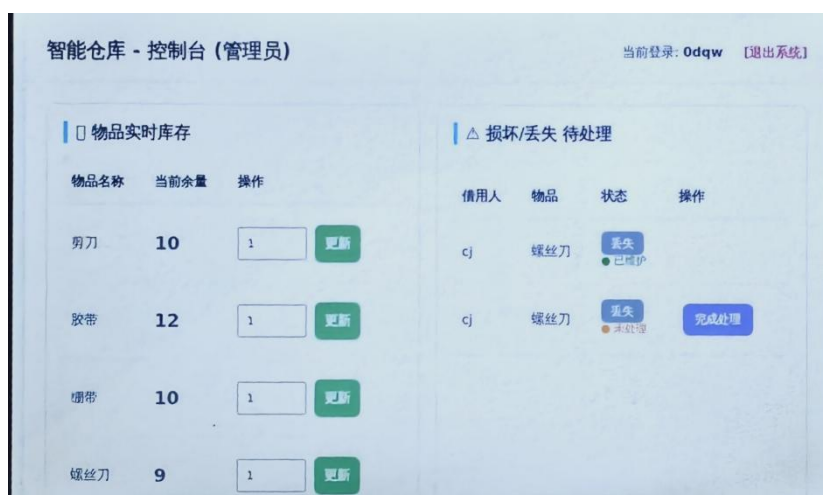


图 3.2.2.6 管理员管理库存、处理损坏丢失界面



图 3.2.2.7 管理员后台历史借还记录界面

3.3 特性成果 (逐个展示功能、性能参数等量化指标)

3.3.1 人脸识别特性成果

采用 RetinaFace 检测定位结合仿射变换实现五点人脸对齐，再输入 MobileFaceNet 提取特征的人脸识别模型能有效识别不同姿态下的人脸。如下列图片所示，在摘戴眼镜、改变发型和改变角度的情况下，均能正确捕捉人脸面部特征，正确识别人脸。

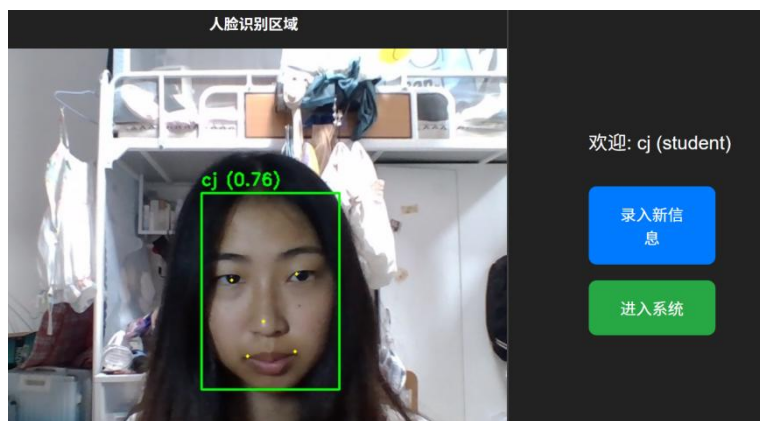


图 3.3.1.1 人脸正常识别

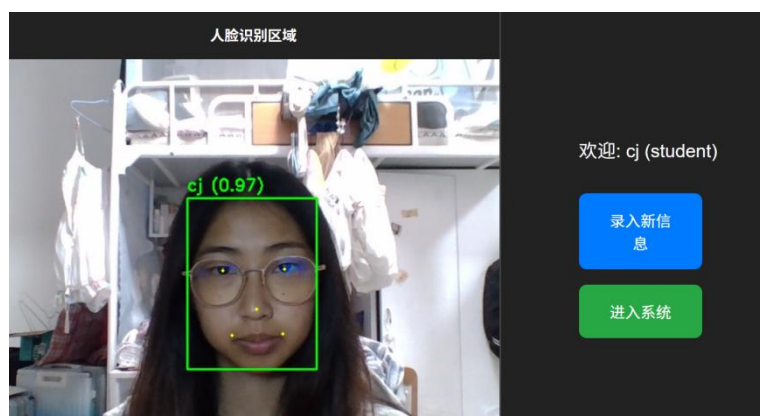


图 3.3.1.2 戴眼镜识别

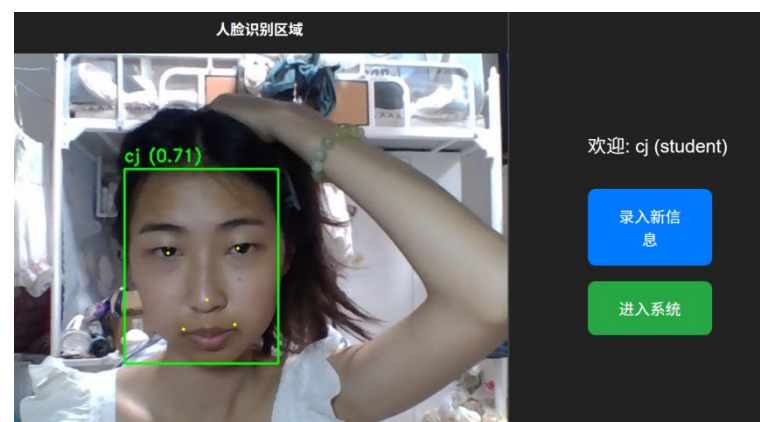


图 3.3.1.3 改变发型识别

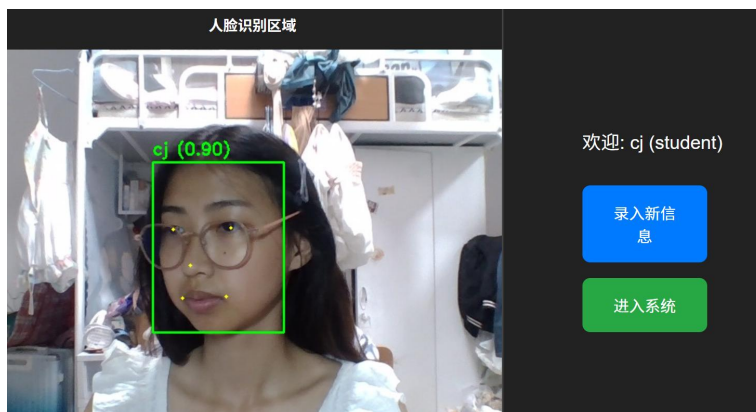


图 3.3.1.4 改变人脸角度正常识别

3.3.2、物品识别特性效果

(1) 自建数据集基本信息

本数据集共包含 980 张真实场景图像，为模拟实际使用场景，图像采集涵盖不同颜色光照以及多角度任意摆放与互相重叠场景，从而提高模型在端侧部署时的泛化能力。

数据集按照 8:2 比例进行随机划分，训练集 777 张，验证集 203 张。

(2) 物品识别模型量化评估:

YOLOv8s 模型在自建宿舍工具数据集上训练 200 个 epoch，训练/验证 box_loss、cls_loss、dfl_loss 均随迭代平稳下降并收敛，Precision(B)、Recall(B) 收敛至约 0.97 与 0.99，mAP50(B) 收敛至约 0.98，mAP50-95(B) 收敛至约 0.675，训练过程未见明显过拟合，如图 3.3.2.1 所示。

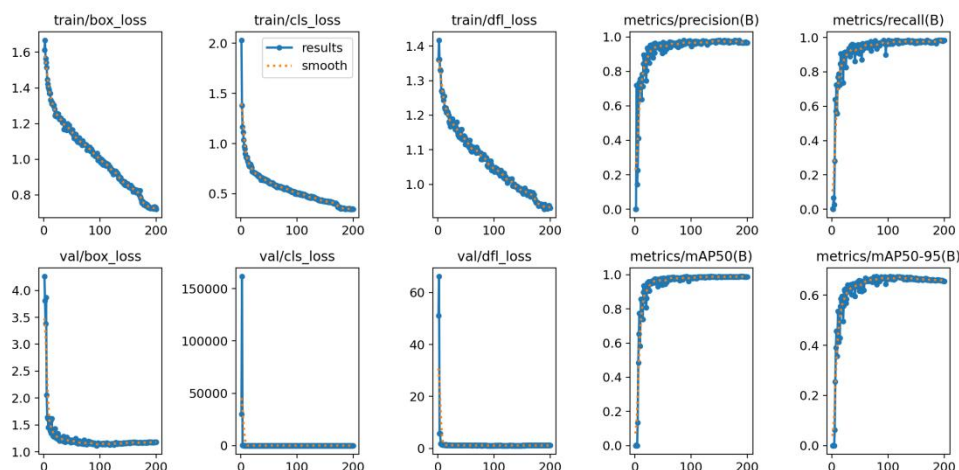


图 3.3.2.1 YOLOv8s 训练/验证损失与精度收敛曲线（200 epoch）

(3) 物品识别实测特性效果分析:

根据验证集随机批次推理可视化结果，图中目标识别物品边界框回归精准，

且单目标置信度普遍保持在 0.8、0.9 以上，体现出本模型体现出高置信度与定位精准度、极佳的抗遮挡与重叠鲁棒性、多姿态与光照自适应优异特性。

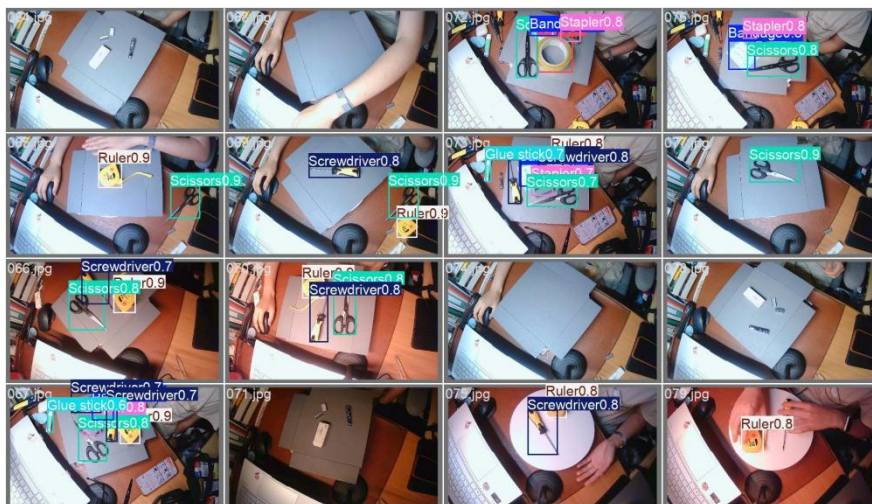


图 3.3.2.2 物品识别实测特性效果

第四部分 总结

4.1 可扩展之处

系统未来将进一步丰富数据集以扩展支持识别的物资与工具品类；同时，计划利用 MQTT/HTTP 协议将本地数据安全上云并对接微信小程序，方便用户远程实时查询库存与到期追溯，实现更高效、精细的移动端流转闭环管理。

4.2 心得体会

本项目的创意源于解决宿舍公共工具管理“无人值守、无据可查”的长期痛点。我们希望通过软硬件协同设计，在成本可控的边缘平台上，实现以往难以在端侧部署的多模型并行处理与可信借还溯源。

实现过程挑战与收获并存。最核心的难点在于异构平台的并发调度与量化精度挽回：一方面，针对 YOLOv8 进行 INT8 量化时出现的检测框偏移问题，我们通过重新校准数据集并优化底层 DFL 解码的数值稳定性挽回了精度损失；另一方面，为解决双路视频流与人脸/物品检测并发竞争 DPU 资源的冲突，我们引入了互斥锁机制与“状态锁定 + 2.5s 容错窗口”状态机，彻底消除了进程阻塞与界面闪烁问题。

这段经历让我们深刻体会到，嵌入式 AI 项目成功的关键，不仅在于算法模型的精度，更在于对硬件算力资源的精细化并发调度，以及面向异常边界的系统

级鲁棒性设计。从深度学习模型压缩到边缘设备上的高可用稳定运行，每一步都是对软硬件协同工程化能力的全面锻炼。

第五部分 参考文献

- [1] 飞凌嵌入式. ELF-RV1126B 开发板快速使用手册: Rev. 1.0 [EB/OL]. (2025-11-28)[2026-04-26]. <http://www.elfboard.com/>.
- [2] 飞凌嵌入式. 进阶篇之-基于 RK 平台的 AI 模型训练到部署: Rev. 1.1 [EB/OL]. (2025-11-25)[2026-04-26]. <http://www.elfboard.com/>.
- [3] FARHAN M. How I deployed YOLOv8 at 30ms on edge hardware (no GPU required)[EB/OL].(2025-06-20)[2026-07-06].<https://github.com/Datraxa/yolov8-rknn-edge>.
- [4] THAINIMIT S, KITIWOURANON K, BUAPHUAN A, et al. Enhanced YOLO-based models with progressive quantization for real-time edge device deployment[J]. Sensors, 2025, 25(4): 1024-1031.
- [6] DENG J, GUO J, ZHOU Y, et al. RetinaFace: Single-stage dense face localisation in the wild[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). Seattle: IEEE, 2020: 5203-5212.
- [6] LIU B, YU H. A lightweight and accurate face detection algorithm based on RetinaFace[J]. arXiv preprint arXiv:2104.03214, 2021.
- [7] ROLAND M. Face to face with efficiency: real-time face recognition pipelines on embedded devices[J]. IEEE Embedded Systems Letters, 2023, 15(2): 88-91.
- [8] ZEDIOT. Deploying YOLOv8 on RK3566: a deep dive into model conversion, quantization, and real-world optimization[EB/OL]. (2025-11-17)[2026-07-06]. <https://medium.com/@zediot>.
- [9] GLASGOW CALEDONIAN UNIVERSITY. Real-time YOLO across cloud, edge, and IoT: architectures, optimisations, and deployment patterns[J]. IEEE Internet of Things Journal, 2026, 15(6): 1120-1135.
- [10] LI B, YUAN H, WANG X, et al. A MobileFaceNet-based face anti-spoofing algorithm for low-quality images[J]. Applied Sciences, 2024, 14(14): 6032-6045.
- [11] LIU X, LIU Y, CHEN Y, et al. LittleFaceNet: a small-sized face recognition method based on RetinaFace and AdaFace[J]. Journal of Sensor and Actuator Networks, 2024, 13(4): 50-65.